

Reliability Assessment of Early Exit Deep Convolutional Neural Networks

Georgios Konstantinidis, Maria K. Michael, and Theodoris Theodoridis

KIOS Research and Innovation Center of Excellence

Department of Electrical and Computer Engineering, University of Cyprus, Nicosia, Cyprus

Email: {konstantinidis.georgios, mmichael, ttheodoridis}@ucy.ac.cy

Abstract—Machine learning inference deployed on edge devices is subject to limited resources, which pushes for better energy and resource efficiency, while ensuring high performance. At the algorithmic level, dynamic Deep Neural Networks (dDNNs) have been proposed in an effort to improve performance and energy efficiency which is particularly useful in time-critical systems such as autonomous navigation and search and rescue operations. These systems are often deployed in harsh environments, making the hardware vulnerable to soft errors. This work investigates the reliability of such dDNNs, specifically convolutional, focusing on early exit approaches, in an effort to better comprehend the impact of the early exits on the overall reliability of dDNNs. We consider transient faults and use hardware-aware software fault models, combined with a fine-grained fault taxonomy, to derive failures-in-time (FIT) rate and assess reliability. We observe a new category of faults specific to dDNNs and we refer to them as non-materialized faults. In our assessment, we use a new fault taxonomy that is more fine-grained than existing ones, also including the new category of non-materialized faults. We explore various early exit scenarios, having as baseline the Google Inception V1 network deployed on an NVDLA accelerator to generate FIT rates and analyze the faults per network architecture block and operation type. Furthermore, we investigate the impact of faults on the overall performance (accuracy) and execution time (average inference time) of the neural network models. Based on our analysis we observe that the overall FIT rate remains similar among different early exit implementations (8.40-8.89), while the distribution of the critical faults is influenced by the network's design-time decisions on early exit block placement and architecture.

Index Terms—Dynamic DNNs, early exit DNNs, edge AI/ML, fault injection, DNN reliability, software-level fault models.

I. INTRODUCTION

Early exit networks are a category of dDNNs capable of adapting their decision path and exit point during inference time [1]–[3]. Such networks are typically optimized for performance and energy, making them excellent candidates for deployment on edge devices. They are trained to allow for multiple execution paths, which in turn manifest to a dynamic decision on a particular exit path during inference time [2]. As with other AI/ML algorithms deployed on edge devices, dDNNs may exhibit run-time failures due to hardware-induced faults, such as transient faults. This occurs particularly when edge devices are deployed in harsh environments, due to high temperature, extraterrestrial radiation, heavy ions from space,

etc. Therefore, in addition to optimizing for performance and energy, it becomes necessary to also analyze the reliability of such networks, to decide if necessary mitigation is required. Mitigation can come in various forms, either via fault-aware training [4], or co-optimization of performance-energy-reliability during design time, or some form of fault tolerance in the network architecture and/or the underlying hardware.

Various existing works conduct reliability analysis and enhancement on DNNs, at software/hardware level or a combination of both. However, such networks are often static, lacking the complex behavior of a dynamic multi-exit network [5]–[7]. Dynamic networks, such as early exit networks introduce dynamic behavior that increases complexity, requiring a comprehensive analysis to better assess energy efficiency, performance (accuracy) degradation, and inference time. The evaluation of such networks involves the combination of metrics calculated dynamically (during inference), such as performance, average inference time, fault impact and failures-in-time (FIT) rate, taking into consideration all possible exit points of an early exit network. To our knowledge, no prior work explores how early exit placement affects network reliability. We address this gap by investigating the interplay between reliability, inference time, and classification accuracy across varying exit placements in convolutional dDNNs.

The contributions of this work are summarized as follows:

- We develop a reliability analysis framework that accounts for the dynamic nature of early exit networks, utilizing hardware-aware software-based fault models for our analysis.
- We show the effects of the impact of soft errors on the Inception V1 network and its dynamic multi-exit variants, taking into account the non-materialized faults and considering our fine-grained fault taxonomy.
- From our analysis we can conclude that even though the average FIT rate appears to remain comparable between various multi-exit variants, the distribution of critical faults within the network changes based on the number, location, and architecture of the early-exit blocks.

The rest of the paper is organized as follows. Section II presents background theory and work related to DNNs and hardware reliability assessment. Sections III and IV describe the proposed methodology and discuss the obtained results, respectively, while Section V concludes the paper.

II. BACKGROUND AND RELATED WORK

A. Background Theory

Faults in hardware accelerators can degrade system reliability and performance, necessitating accurate fault detection and classification into permanent, transient, or intermittent types. Modern reliability assessment methods rely on fault modeling implemented via software, hardware, or hybrid approaches. These methods vary in complexity based on the underlying neural network architecture and the hardware platform. One of the most widely adopted methods is fault injection, where faults are injected on various positions of the system (either hardware or software) [6]. Dynamic deep neural networks (dDNNs), such as the one shown in Fig. 1 introduce additional complexity compared to static ones due to their input-dependent execution paths. The process of reliability assessment requires observability and controllability capabilities of various network modules. In dDNNs, early exits are taken probabilistically based on input and exit policies. Thus, the computation graph becomes dynamic, resulting on traversing the vertices of the network in a non-deterministic way.

Since exit decisions are guided by subsequent feature maps from the graph, the reliability assessment should be treated in a dynamic way, a functionality that is not included in standard frameworks [1]. An example of complexity that stems from the dynamic behavior of dDNNs indicates the existence of untraversed paths on a dDNN that may never be activated during inference [2]. We observe that this behavior leads to the so called non-materialized faults during fault injection campaigns. These faults can affect the statistical significance of a fault injection campaign, as they are never materialized due to the dynamic nature of the inference process, however, they may still be counted as injected faults. To address this issue, we use a fine-grained fault taxonomy, shown in Fig. 3, which also distinguishes between faults that affect the classification output and those that only influence prediction confidence.

B. Related Work

The reliability assessment of neural networks has been typically handled via fault injection. Each fault injection technique, either software-based injection, hardware emulation, or radiation testing, provides trade-offs in simulation speed, accuracy, and development effort [5]–[7]. Recent trends favor hybrid approaches to balance these factors. At the software level, frameworks like [8] and PyTorchFI [9], enable fault injection in DNNs for both resilience analysis and model interpretability. Other efforts have developed hardware-aware software fault models that incorporate microarchitectural features to improve simulation realism and efficiency [10]. Emulation-based studies such as [11], analyze fault propagation on the NVDLA accelerator using RTL models in a cross-layer manner, while [12] analyzes a tiny TPU-inspired architecture using FPGA emulation for fast yet accurate fault injection campaign. In [13], the authors propose a runtime fault detection mechanism that operates on-the-fly during inference.

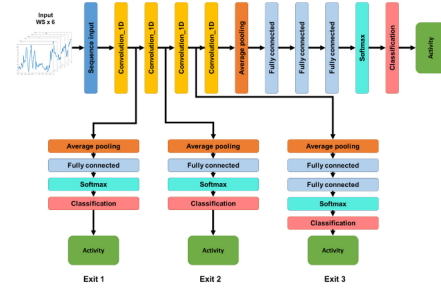


Fig. 1: Example of early exit CNN [18] with 3 early exits. An early exit block can include a series of average pooling, fully connected and softmax layers, followed by classification.

It computes check-sums dynamically alongside normal data flow, enabling real-time detection of faults in systolic arrays. The work in [14] compares software and hardware-level fault injection, finding that ISA-level injection more accurately reflects real-world faults, especially in safety-critical applications. Fault analysis also extends to spiking neural networks, as shown by [15] and [16], which introduce fault injection platforms for SNNs. In their work, they briefly discuss the trade-offs between gate, RTL, and microarchitectural level fault injection and how the speed and accuracy of simulation increases and decreases, respectively, as the abstraction level increases. Despite these advances, reliability evaluation of dynamic networks remains largely unexplored. The authors in [17] consider split-computing which deviates from traditional static models, however, the execution paths remain static even though they are split. Due to the input-dependent execution paths in dDNNs, fault effects can vary significantly impacting both accuracy and inference time. A comprehensive analysis must re-consider the FIT rate calculation to account for all exit points, dynamic path activation, and performance degradation under fault conditions to accurately characterize them. In this work, we address this gap by conducting a reliability analysis of dDNNs using hardware-aware software-based fault injection, as trade-off between accuracy, speed, and hardware complexity, capturing the impact of the dynamic behavior of the network on the overall reliability.

III. PROPOSED METHODOLOGY

A. Fault Injection Framework Architecture

To effectively investigate the reliability of early exit networks we implemented the framework illustrated in Fig. 2. The framework was specifically tailored for the characteristics of early exit networks, providing metrics related to prediction accuracy, average inference time, and how both are influenced via the position and number of exit branches as a function of reliability. The proposed reliability evaluation is divided to three phases. In the first phase, we import into the framework the trained parameters and graphs of the network. To determine the exit strategy of the branches, various exit criteria can be applied as proposed in [1], as the framework allows for that. The exit threshold values can be derived based on multiple inferences of input data and observations of the statistical characteristics (mean value, standard deviation) from

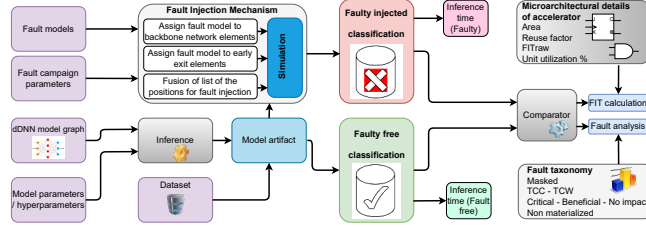


Fig. 2: Proposed framework for dDNN fault analysis using fine-grained fault taxonomy and microarchitectural characteristics for FIT calculation.

TABLE I: Utilized fault models, extended from [10].

Fault Model	Description
Inputs	Single bit fault at an input of a neuron.
Weights	Single bit fault at a weight.
Multiple Inputs	Single bit fault at an input affecting N consecutive neurons.
Multiple Weights	Single bit fault at a weight affecting N consecutive neurons.
Output	Single bit fault at the output of a neuron.
Control Variables	Random value fault at the output of a neuron
Early Exit Blocks	Input, Weight, Multiple Input, Multiple weight, Output and Control variable faults in an early exit block.

the selected exit points. The network parameters are then integrated into the network layers to structure dictionaries that store detailed information about the network internal operations, feature map dimensions, and the positional context of each layer within the architecture. The framework allows customization of the network graph in terms of architecture, exit policies, and operations. The fault campaign is fused by integrating the dynamic network graph and the generated dictionaries combined with the fault models and early exit point dictionaries.

These components and data feed the fault injection mechanism, which generates a list of potential injection positions. This fused list is further augmented with a defined set of parameters such as fault distribution, desired confidence, error thresholds, and the associated bit positions to guide the injection process effectively. Alongside the testing dataset, a set of network-specific information is provided to a mechanism responsible for determining the precise injection positions. In this work, the single bit-flip fault model is employed (transient faults are considered). During the third and final stage, the fault injection campaign is executed and analyzed. The network under evaluation is supplied with the appropriate test dataset, and faults are systematically injected. The inference mechanism generates for each input both a fault-free and a fault-injected prediction. These outputs are then compared, and the results are correlated with the micro-architectural characteristics of the underlying accelerator. Subsequently, the FIT rate is calculated, faults are categorized, and the impact on accuracy and execution time is evaluated. Notably, the fault injection campaign is applied in a static manner, while the operational behavior of the network remains dynamic. This discrepancy leads to the emergence of a new category of faults, which is thoroughly examined in Section III-C.

B. Fault Models

The choice of the utilized fault models (Table I) affects the credibility of the fault injection campaign, with hardware-aware fault models typically providing better accuracy compared to purely software-level models. To effectively execute such a fault campaign the fault models were adopted from [10]. The models target various components of the neural network associated with input, weight, output feature maps, and feature maps that affect consecutive hardware elements such as output buffers, N datapath buses, etc. The fault models are associated with the hardware that hosts the neural network, making them hardware-aware. We introduce a new category of fault model in the fault model suite, on top of the six fault models proposed by [10], aiming to study the early exit blocks in dDNNs. The new category makes use of the existing fault models to create a representative campaign that targets modules in early exit blocks. A brief description of the fault models is provided in Table I.

C. Proposed Fault Taxonomy and Non-Materialized Faults

To analyze dDNN behavior when different exit points are introduced, this study employs an error taxonomy alongside a revised FIT metric for quantitative evaluation. While existing taxonomies (e.g., [7], [15]) classify faults primarily at the DNN level based on changes in classification probabilities, labeling a fault as unsafe if it alters the final result and safe otherwise, this work adopts a more fine-grained, system-level approach. This refined taxonomy, inspired by [12], considers both changes in classification outcomes and shifts in confidence scores relative to fault-free inference. It also takes into consideration the so called non-materialized faults that affect the confidence and error of the campaign.

The proposed fault taxonomy is shown in Fig. 3. Faults are categorized as follows:

- Critical (C): Negative change in decision.
- Beneficial (B): Positive change in decision.
- No impact (NI): "Don't care" change in decision.
- Tolerable classified correct (TCC): Change in confidence scores but not on decision.
- Tolerable classified wrong (TCW): Same as TCC but for misclassifications.
- Masked (M): No change on decision and confidence scores.

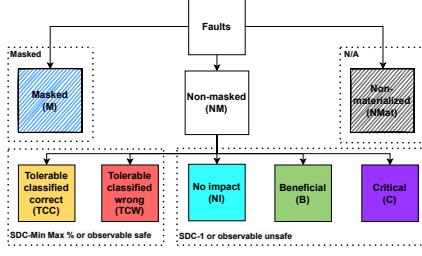


Fig. 3: Fault taxonomy utilized for our reliability analysis, showing the correlation with SDC min-max and SDC-1 categories of [15], and observable safe and unsafe of [7].

- **Non-materialized (NMat):** Introduced in this study to capture faults that never materialize due to the dynamic nature of dDNN inference termination decisions.

These non-materialized faults highlight the mismatch between static fault selection/injection and the dynamic execution paths of early exit networks, which vary in computation time and fault exposure. Moreover, they can potentially affect the confidence level and/or error % of a fault campaign, therefore they need to be quantified carefully. Although a coarse-grained fault taxonomy, as the ones presented in [7] and [15], could also be employed, this more fine-grained taxonomy which can potentially unveil a smaller number of critical faults, can better guide system-level evaluations where a critical fault has to be propagated to the application level.

D. FIT Rate Calculation

We consider microarchitectural characteristics of the accelerator for the FIT calculation. The formula used to calculate the FIT rate for a dDNN is given in (1). The first term accounts for the FIT of the backbone exit point (single exit), while the second term considers the possible early exit points (indexed by j). As in [10], FIT calculation is only related to the non-masked faults. In (1), m is the number of the different fault models as described in Table I, excluding the early exit block (i.e., $m = 6$). n is the number of early exit points in a dDNN, RF is the reuse factor utilization (flip flop associated) for the corresponding fault model, NM denotes the percentage of the non-masked faults, FIT_{raw} is described by the underlying technology (manufacturing process), denoting the flip flop FIT raw rate. The % of the area occupied on the accelerator die for a fault model is denoted as A_i^{se} for the backbone network and A_j^e for an early exit. Similarly, TE_{se} and TE_j denote the utilization % for the backbone and each early exit, respectively.

$$FIT = \sum_{i=1}^m RF_i \times NM_i^{se} \times FIT_{raw} \times A_i^{se} \times TE_{se} + \sum_{j=1}^n \sum_{i=1}^m RF_j^i \times NM_i^j \times FIT_{raw} \times A_j^e \times TE_j \quad (1)$$

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

We consider Inception V1 convolutional DNN as baseline network and generate three early exit variants as shown in Fig. 4. We leverage the moderate depth of Inception V1 to

TABLE II: Faults breakdown as given in Fig. 3 for the considered networks in Fig. 4.

	V1 Backbone (Net1)	Shallow (Net2)	Deep (Net3)	Multi-exit (Net4)
Critical (C)	4.2 %	3.8 %	2.9 %	3.1 %
Beneficial (B)	0.2 %	0.5 %	0.3 %	0.6 %
No impact (NI)	0.5 %	0.9 %	0.8 %	0.8 %
SDC 1 / Observable unsafe [7] [15]	4.9 %	5 %	4 %	4.5 %
Tolerable classified correct (TCC)	7.5 %	7.8 %	7.2 %	7.4 %
Tolerable classified wrong (TCW)	2.1 %	2.3 %	2.7 %	2.8 %
SDC Min-Max % / Observable safe [7] [15]	9.6 %	10.1 %	9.9 %	10.2 %
Masked (M)	85.5 %	67.7 %	75.6 %	66.6 %
Non-materialized (NMat)	-	17.2 %	10.5 %	18.7 %
Total	100 %	100 %	100 %	100 %

TABLE III: Fault free (FF) and faulty injected (FI) exit utilization for the considered networks in Fig. 4. Accuracy and FIT rates are also reported.

	Exit		Shallow exit		Deep exit		Accuracy		FIT
	FF	FI	FF	FI	FF	FI	FF	FI	
Net1	100%	100%	-	-	-	-	80%	76%	8.88
Net2	75%	73%	25%	27%	-	-	76%	73%	8.89
Net3	65%	60%	-	-	35%	40%	75%	72%	8.46
Net4	54%	48%	25%	27%	21%	25%	74%	71%	8.40

explore meaningful early exit strategies avoiding placement of exits within the middle of inception blocks and employ a fault model targeting also these early exit components. Fig. 4 (Net1) illustrates the primary architectural blocks of the Inception V1 backbone, which comprises convolutional layers, average pooling, and fully connected layers. For our experiments, we evaluated three different early exit networks based on Inception V1, as shown in Fig. 4. The first two networks (Net2 and Net3) implement single early exit points, while Net4 incorporates multiple early exits. Furthermore, we consider the NVDLA accelerator for the network evaluation. Based on its microarchitecture, we set the parameter $N=16$ for the models in Table I. The raw flip-flop FIT rate used is 600 FIT/MB [19]. The reuse factor in (1) reflects the frequency of flip-flop utilization for each fault model during inference.

The utilized framework was implemented in Python. The dataset used for the training and testing of the networks is CIFAR-10. The networks were trained using the method of stochastic gradient descent with a learning coefficient equal to 0.0001 and a batch size of 64. The fault injection campaigns for the four networks in Fig. 4 were fused in a stratified way to ensure statistical correctness, using the formula from [20], which accounts for all possible injection positions (including early exits). Each fault model maps software variables to hardware instances, as detailed in Table I. The fusion considers the dynamic structure of the networks, including backbone and early exits. For each network variant 42 000 faults were injected, yielding 99% confidence and 1.5% error margin.

B. Results

In Table II, we list the % of faults in each category of our fine-grained taxonomy given in Fig. 3, as well as the ones under the taxonomies in [7], [15]. In our taxonomy, the % of critical faults is reduced significantly compared to [7], [15], especially for the early exit networks (ranging between 25.5%–31%), due to the identification of beneficial and no impact faults. Non-materialized faults in the dDNN variants range between 10.5%–18.7%, affecting the overall number of realized injected faults during the fault campaign. Using the statistical formula of [20], this modifies the intended 1.5% error margin to 1.65% (Net2), 1.59% (Net3), and 1.67% (Net4), while maintaining the confidence level to 99%. Hence,



Fig. 4: Architectures of the four networks utilized for reliability analysis: Net1 is the original Inception V1 model, Net2 incorporates shallow early exit block, Net3 is trained with deep early exit point, and Net4 is a multiple early exit network.

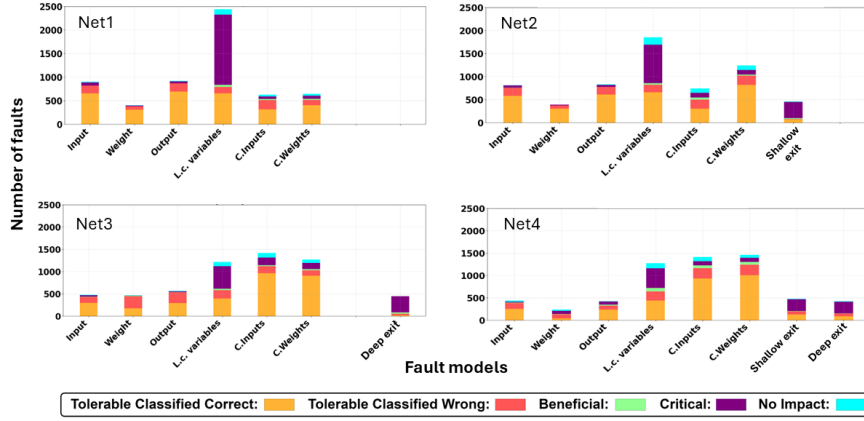


Fig. 5: Fault analysis per considered fault model for the baseline (Net1) and early exit versions (Net2, Net3, and Net4) of Inception V1, based on the proposed fault taxonomy (masked and non-materialized faults are excluded for readability purposes, but provided in total in Table II).

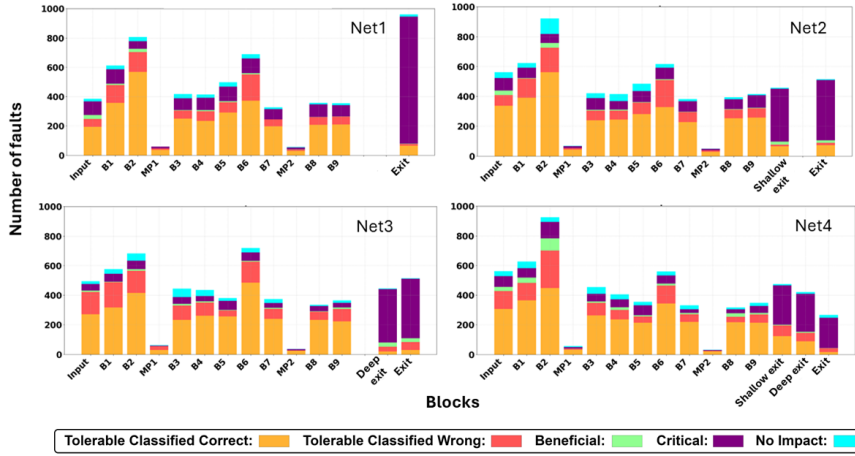


Fig. 6: Fault analysis per network architecture block for the baseline (Net1) and early exit versions (Net2, Net3, and Net4) of Inception V1, based on the proposed fault taxonomy (masked and non-materialized faults are excluded for readability purposes, but provided in total in Table II).

the campaign results remain statistically significant. If this was not the case, then additional faults should have been injected.

The performance accuracy, FIT rate, and exit % per model are given in Table III. Overall, FIT rates remain similar, but as shown in Fig. 6, the distribution of the faults (especially the

critical ones) changes significantly. Performance accuracy suffers a drop between 3–4% in the fault-injected (FI) networks, compared to the fault-free (FF) ones. The average inference time, which can be significantly affected in dDNNs due to the variant exit utilization, is reduced by 13.3% (Net2), 7.9%

TABLE IV: Breakdown of faults for taxonomy in Fig. 3 based on network operation.

Net	Operation	M (%)	TCC (%)	TCW (%)	B (%)	C (%)	NI (%)	NMat (%)
Net1	Conv	74.7	4.8	1.3	0.1	0.8	0.2	-
	Maxpool	5.3	1.2	0.3	0.1	0.5	0.2	-
	Avg.Pool	0.6	0	0	0	0.6	0	-
	Concat	4.4	1	0.5	0	0.8	0.1	-
	Fully connected	0.9	0.1	0	0	1.5	0	-
Net2	Conv	59.5	4.8	1.5	0.2	0.5	0.5	9.2
	Maxpool	2.9	1.6	0.4	0	0.4	0.2	3.5
	Avg.Pool	0.5	0.1	0	0	0.6	0	0.4
	Concat	3.7	1.2	0.3	0	0.9	0.2	3.4
	Fully connected	1.1	0.3	0.1	0.1	1.2	0	0.7
Net3	Conv	64.9	2.5	0.9	0.1	0.5	0.3	5.3
	Maxpool	5.4	2.1	0.6	0	0.4	0.3	0.1
	Avg.Pool	0.4	0	0.1	0	0.6	0	0.5
	Concat	3.9	2.5	1	0.1	0.3	0.2	3.6
	Fully connected	1	0.1	0.1	0.1	1.2	0	0.9
Net4	Conv	49.3	5.8	0.8	0.4	0.3	0.1	15.3
	Maxpool	7.1	0.4	0.8	0.1	0.4	0.3	0.6
	Avg.Pool	0.8	0.1	0.1	0	0.6	0	0.6
	Concat	7.7	0.6	0.7	0.1	0.6	0.3	0.8
	Fully connected	1.7	0.4	0.3	0	1.1	0.1	1.5

(Net3), and 18% (Net4), compared to the average inference time of the backbone static network (Net1). This is clearly in accordance to one of the main goals of dDNNS, which is the reduction of the average inference time.

Fig. 5 provides our fault analysis per fault model according to Table I, showing that critical faults are much more prominent when there is a fault in the local variables, mainly appearing in exit blocks, both original and early exits. This is shown more clearly in Fig. 6, which provides the same analysis, but per architectural block in the baseline Inception V1 and its early exit variants. We can observe that critical faults are distributed among all possible exits. Table IV summarizes the fault distribution based on the network functional operations. Convolution is clearly the most resilient operation, exhibiting the majority of masked faults, whereas fully-connected layers seem to suffer more from critical faults. Non-materialized faults can occur in any operation, but predominantly in convolution.

C. Observations and Discussion

Early exit network behavior is hard to predict due to dynamicity caused by input dependency. It can also potentially impact the fault campaign accuracy due to generation of non-materialized faults. From our observations for the Inception V1, the introduction of early exit points reduces the vulnerability of the original static exit but increases vulnerability on early exit blocks, while the overall FIT rate seems to not be significantly affected. We observe that all networks are vulnerable at their initial layers. Fig. 6 and Table IV indicate that the initial layers of the network are more susceptible to small perturbations due to high percentage of tolerable faults. The final layers are the most vulnerable, with fully connected and concatenation operations being the most vulnerable. The mixture of masked and non-critical faults indicates a substantial resilience in intermediate convolutional layers. Although the overall reliability of the network seems to slightly increase with added exits, every exit block is highly vulnerable to faults.

V. CONCLUSIONS AND FUTURE WORK

In this study, we investigate the reliability of Inception V1 by constructing dynamic variants of the model subjected to fault injection, and analyze how the placement of early exits affects accuracy, inference time, and reliability. This was achieved via the use of a newly fine-grained adopted

taxonomy and the adjusted calculation of FIT rate for each network. Future work will investigate mitigation approaches, applied based on the identified critical layers and parameters, as well as neural architecture search for co-optimizing the performance-energy-reliability of early exit models.

ACKNOWLEDGMENT

This work has been supported by the European Union's Horizon 2020 research and innovation programme under grant agreement No. 739551 (KIOS CoE) and from the Government of the Republic of Cyprus through the Cyprus Deputy Ministry of Research, Innovation and Digital Policy.

REFERENCES

- [1] S. Laskaridis, A. Kouris, and N. D. Lane, "Adaptive inference through early-exit networks: Design, challenges and directions," in *Proc. of the 5th International Workshop on EMDL*, 2021, pp. 1–6.
- [2] J. Jo, G. Kim, S. Kim, and J. Park, "Locoexnet: Low-cost early exit network for energy efficient cnn accelerator design," *IEEE Trans. CAD Integr. Circuits Syst.*, vol. 42, no. 12, pp. 4909–4921, 2023.
- [3] H. Rahmath *et al.*, "Early-exit deep neural network-a comprehensive survey," *ACM Comput. Surv.*, vol. 57, no. 3, pp. 1–37, 2024.
- [4] F. F. dos Santos, N. Cavagnero, M. Ciccone *et al.*, "Improving deep neural network reliability via transient-fault-aware design and training," *IEEE Trans. Emerg. Top. Comput.*, 2025.
- [5] M. H. Ahmadilivani, M. Taheri, J. Raik *et al.*, "A systematic literature review on hardware reliability assessment methods for deep neural networks," *ACM Comput. Surv.*, vol. 56, no. 6, pp. 1–39, 2024.
- [6] A. Ruospo, E. Sanchez *et al.*, "A survey on deep learning resilience assessment methodologies," *Comput.*, vol. 56, no. 2, pp. 57–66, 2023.
- [7] A. Ruospo, S. Pappalardo, V. Turco *et al.*, "Reliability of deep neural networks: Impact and open issues," *IEEE Des. & Test*, 2025.
- [8] A. Ruospo, M. S. Reorda, R. Mariani *et al.*, "An effective iterative statistical fault injection methodology for deep neural networks," *IEEE Trans. Comput.*, 2025.
- [9] A. Mahmoud *et al.*, "Pytorchfi: A runtime perturbation tool for dnns," in *Proc. of the IEEE/IFIP International Conference on DSN-W*, 2020, pp. 25–31.
- [10] Y. He, P. Balaprakash, and Y. Li, "Fidelity: Efficient resilience analysis framework for deep learning accelerators," in *Proc. of the 53rd Annual IEEE/ACM International Symposium on MICRO*, 2020, pp. 270–281.
- [11] A. Veronesi, A. Nazzari, D. Passarello *et al.*, "Cross-layer reliability analysis of NVDLA accelerators: Exploring the configuration space," in *Proc. of the IEEE ETS*, 2024, pp. 1–6.
- [12] P. Cornelio, P. Nikolaou, M. K. Michael, and T. Theocharides, "Fine-grained vulnerability analysis of resource constrained neural inference accelerators," in *Proc. of the IEEE International Symposium on DFT in VLSI and Nanotechnology Systems*, 2021, pp. 1–6.
- [13] W.-K. Liu, J. Talukdar, B. Tan *et al.*, "Effective runtime fault detection for DNN accelerators," in *Proc. of the IEEE ATS*, 2024, pp. 1–6.
- [14] G. Esposito, J.-D. Guerrero-Balaguera *et al.*, "Evaluating different fault injection abstractions on the assessment of DNN SW hardening strategies," in *Proc. of the IEEE ATS*, 2024, pp. 1–6.
- [15] A. B. Gögebakan *et al.*, "Spikingjet: Enhancing fault injection for fully and convolutional spiking neural networks," in *Proc. of the IEEE IOLTS*, 2024, pp. 1–7.
- [16] T. Spyrou, S. Hamdioui, and H.-G. Stratigopoulos, "Spikefi: A fault injection framework for spiking neural networks," *arXiv preprint arXiv:2412.06795*, 2024.
- [17] G. Esposito, J.-D. Guerrero-Balaguera *et al.*, "Enhancing the reliability of split computing deep neural networks," in *Proc. of the IEEE IOLTS*, 2024, pp. 1–7.
- [18] E. Lattanzi, C. Contoli, and V. Freschi, "Do we need early exit networks in human activity recognition?" *Eng. Appl. Artif. Intell.*, vol. 121, 2023.
- [19] S. Jagannathan, T. Loveless *et al.*, "Frequency dependence of alpha-particle induced soft error rates of flip-flops in 40-nm CMOS technology," *IEEE Trans. Nucl. Sci.*, vol. 59, no. 6, pp. 2796–2802, 2012.
- [20] R. Leveugle, A. Calvez, P. Maistri *et al.*, "Statistical fault injection: Quantified error and confidence," in *Proc. DATE*, 2009, pp. 502–506.